Andrew Collins



I am an adaptable Robotics M.S. student and researcher at Georgia Tech with a background in mechanical engineering. I am passionate about enabling robots to safely interact with the real world via touch, and experienced in developing intelligent hardware and software solutions spanning autonomous navigation, multimodal sensing packages, end-to-end deep learning pipelines, and robust control systems. I specialize in tactile sensing and contact rich interaction for humanoid robots and am interested how robots with tactile sensors can collaborate with human partners.

Website · **LinkedIn** · **GitHub**

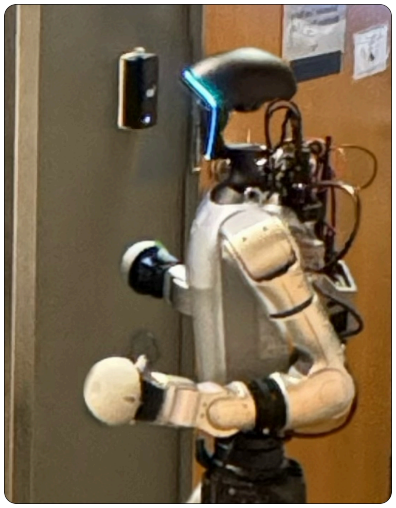
Publications

TAPNAV

Under Review for ICRA 2027

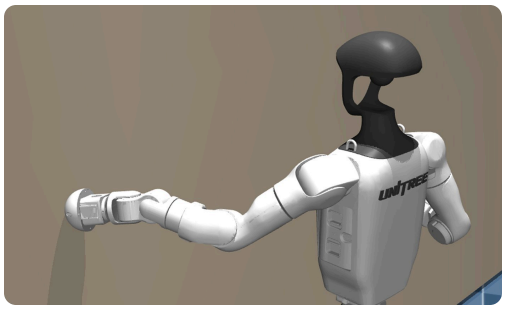
Huaze Liu, Zhenyu Wu, Jaehwi Jang, Junjie Sheng, **Andrew Collins**, Aaron Xie, Zhaoyuan Gu, Kaijie Zhu, Ding Jiang, Kevin Cai, Ye Zhao

TLDR: TAPNAV is a humanoid navigation pipeline that enables robots to reason about their environment tthrough touch. By probing surfaces TAPNAV builds a picture of a robots pose uncertainty, then decides whehter or not it is safe to proceed. Global planning also accounts for uncertainty, so the robot both plans for uncertainty and accounts for it at test time. My contribution was to early hardware development, environment simulation, and constrained uncertainty propagation.



TAPNAV is a robotics navigation pipeline that allows humanoid robots to feel their way around their environment with strategic contacts. A global planner plans a route with the lowest amount of predicted uncertainty that also minimzies distance. At test time, TAPNAV propogates uncertainty when the robot is moving, then reduces it when contact are made with walls and obstacles. Whole body control is achieved through upper and lower body policies that track end effector pose and foot pose.

We tested TAPNAV on both simulation and real world deployments. One of my contributions was evironment simulation, and ensuring feasibility for navigation. I generated a set of maps that mimicked real world rooms, including curving hallways, tight corridors, and larger more open spaces. I then added objects to these rooms in a pseudo-random fashion, selecting from a set of reasonable points where obstacles might be in real rooms. I included tolerances to ensure that no spaces on the maps were too small for the robot to pass through unless it was intentionally so. Each of these maps had 2D representations for global planning as well



as MuJoCo versions for simulation. These maps were used as part of a study to prove that TAPNAV generalized to various maps and not just the cleaner ones tested in some of our real world experiments.

My other major contribution to TAPNAV was working through the implementation of the uncertainty propagation for the global planner. Without the global planner accounting for uncertainty, the modified A* algorithm would simply take the shortest route. Instead, the planner estimates the amount of uncertainty the real robot would gain when in motion, and propogates that at every time step. It then discards any valid routes with too much uncertainty to help prevent the robot from taking potentially dangerous paths. While the final implementation is not entirely mine (we are a team after all!), I did help with the initial design of the directional-propagation system and.

Project Page

WT-UMI

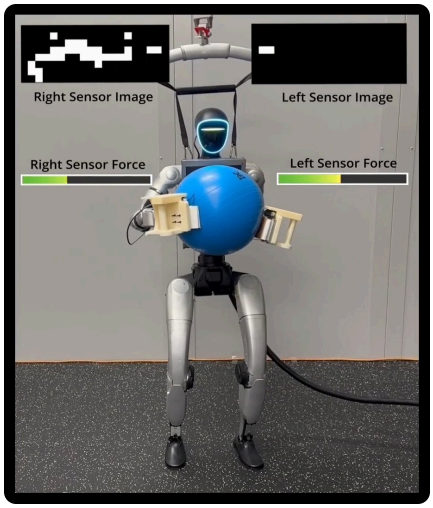
Accepted to CoRL 2026

Jaehwi Jang, Zhaoyuan Gu, Alfred Cueva, Zimeng Chai, Junjie Sheng, Thong Nguyen, Himank Galundia, Yifan Wu, Huishu Xue, Isaac Legene, Ojas Mediratta, Davin Doan, **Andrew Collins**, Sarah Sadegh, KyoungMok Kim, Rishita Dhalbisoi, Zun Chen, Ye Zhao



TLDR: As part of my research for the Georgia Tech LIDAR Group, I worked on a team researching tactile sensing and whole body manipulation. Together we developed a pipeline that enabeled humans to collect data that could be used to train a whole body manipulation planner policy driven by tactile sensor input. I worked predominantly on sensor hardware design and integration.

While working with the Georgia Tech LIDAR group I was able to contribute to a project focusing on using tactile sensors on more than just the hands of humanoid robots, trying to ascertain whether it was useful for improving manipulation control. I contributed to the project hardware and sensing integration. We submitted to CoRL and are waiting to hear back.



We tested the robot on five different tasks throughout the project: yogaball rotation, bucket rotation, pillow rotation, beam carrying, and table carrying. I was directly involved with the design and fabrication of the sensing hardware used throughout the project. We used thin film tactile sensors on modular and removable 3D printed components. This allowed us to have a highly versatile kit of swappable tactile hardware that we could use to cover the hands, chest, and forearms of the UNITREE G1 we were using. I also contributed to the integration of these pieces of hardware into the software used by the team. This included calibration and datagathering about the sensors, developing and tweaking sensor pipelines and drivers, and setting up ROS2 integrations.

The overall system pipleine takes tactile readings, robot sensor data, and target end effector positions. Part of the goal of the project was to use

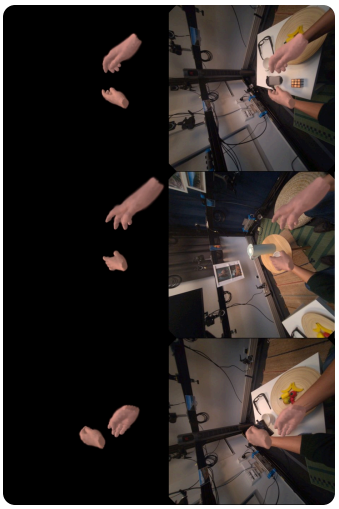
predominantly human-gathered data when training the robot's action planner, so corrections needed to be made to ensure that the correct motions would be completed. A multilayer pretrained pose correction system was implemented to deal with this. The corrected target pose were used in a force-aware planner, that used a ViT encoder and a diffusion policy basis to plan future robot trajectories.

An admittance controller and an IK controller were used to convert the planned trajectories into actual achievable motions, while still being aware of perturbations. Low level control was handled bu joint-level PD controllers. Overall, the final pipeline was successful at proving that adjusted human data could be used to train a control policy, and that tactile sensing was useful for maintaining contact in manipulation tasks.

Project Page

Projects

Hand2Hand



TLDR: Hand2Hand is a video processing pipeline that uses gaussian splatting to build a 3D model of a hand from an input video, similar to the Glove2Hand project. As part of one of the research projects with the Georgia Tech LIDAR Lab, we needed a pipeline similar to the original Glove2Hand paper but with more generalizability to arbitrary videos. My approach used 3D meshes and a small MLP to bake colors to the gaussians. Overall, I was able to deliver a pipeline that successfully created a 3D hand model from an arbitrary input video while being efficient enough to complete recounstructions of the entire Hot3D dataset on a single 4090 in less than one day.

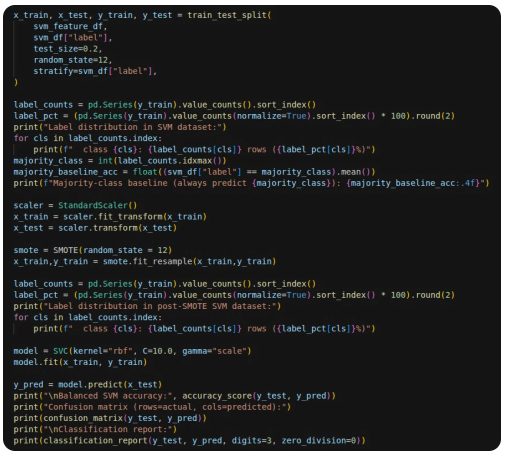
As part of a research project with the Georgia Tech LIDAR Lab, I worked to develop a pipeline to build 3D models of hands from 2D videos. I based my work on the Glove2Hand research project by Zhang et al. from Rutgers University and Meta Reality Labs. Unlike the Glove2Hand pipeline though, I wanted to be able to use arbitrary videos rather than prerecorded videos with high quality data. My approach allowed for two inputs: plain video data that Hand2Hand fit Mano meshes to, or Hot3D recordings, which already had fit 3D Mano meshes. Once the 3D mesh overlay was established, gaussians were propagated at each vertex and on each face. Depending on the size of the face and the density of the vertices in that area, gaussians were given different sizes and quantities to maintain a smooth look. A small MLP was used to bake the colors to the gaussians. Problematically, input videos may contain frames where the hands are out of frame or hidden behind an object. To prevent these bad frames from affecting the color of the hands, I took two approaches;

1. Bake colors using the whole video. The majority color for the hands would be the intended color, so I trained on as much good color data as possible.
2. Reject colors out of the expected range. When unexpected colors did occur (outside 1.5 standard deviations), I replaced the colors of these gaussians with the colors of their k nearest neighbors.

I also had to design the system for computational efficiency, due to the limited compute available. This involved ensuring RAM and VRAM were managed correctly to prevent memory leaks, and that the lifecycle of any running process always cleaned up after itself. At the conclusion of the project, I was able to run the pipeline on the entire Hot3D dataset in less than one day on a single RTX 4090.

Code

Predicting Humor with Machine Learning



```
x_train, x_test, y_train, y_test = train_test_split(
    svm_feature_df,
    svm_df["label"],
    test_size=0.2,
    random_state=12,
    stratify=svm_df["label"],
)

label_counts = pd.Series(y_train).value_counts().sort_index()
label_pct = (pd.Series(y_train).value_counts(normalize=True).sort_index() * 100).round(2)
print("Label distribution in SVM dataset:")
for cls in label_counts.index:
    print(f"  class {cls}: {label_counts[cls]} rows ({label_pct[cls]}%)")
majority_class = int(label_counts.idxmax())
majority_baseline_acc = float((svm_df["label"] == majority_class).mean())
print(f"Majority-class baseline (always predict {majority_class}): {majority_baseline_acc:.4f}")

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

smote = SMOTE(random_state = 12)
x_train,y_train = smote.fit_resample(x_train,y_train)

label_counts = pd.Series(y_train).value_counts().sort_index()
label_pct = (pd.Series(y_train).value_counts(normalize=True).sort_index() * 100).round(2)
print("Label distribution in post-SMOTE SVM dataset:")
for cls in label_counts.index:
    print(f"  class {cls}: {label_counts[cls]} rows ({label_pct[cls]}%)")

model = SVC(kernel="rbf", C=10.0, gamma="scale")
model.fit(x_train, y_train)

y_pred = model.predict(x_test)
print("\nBalanced SVM accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix (rows=actual, cols=predicted):")
print(confusion_matrix(y_test, y_pred))
print("\nClassification report:")
print(classification_report(y_test, y_pred, digits=3, zero_division=0))
```

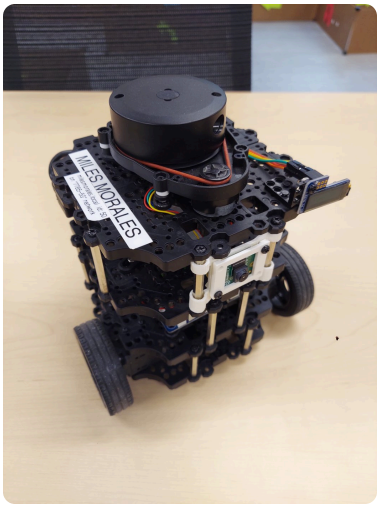
TLDR: For the final project of my Machine Learning course I worked with four other students to apply machine learning to predict whether jokes would be viral. Using a dataset of one-million jokes from the r/jokes subreddit we applied SVM and logistic regression to help us make inferences on unseen jokes. While our final pipeline may have struggled, it was an interesting exploration into the limitations of natural language processing.

Our goal with this project was to take the r/jokes dataset that we had selected and see if we could use it to predict whether a joke would be successful, or viral, using the joke text itself and the Reddit scores associated with them. To improve our chances of getting useful results, we did some simple preprocessing to prepare the entire dataset. This included removing jokes that had missing headings, text, or scores and removing whitespacing, capitals, and stopwords. We saved the number of capitals and punctuation marks in the event they were useful. Each methods used a unique form of tokenization as part of their preprocessing. HDBSCAN used Glove embeddings, since they were already pretrained. SVM and logistic regression used word2vec to extract possible new meanings from the jokes.

The lack of strong results of this project highlight the difficulty of approaching NLP tasks. Future work could include applying multiple layers of models to the data, using new preprocessing methods, or using a different tokenization method. Had the project been successful, we could feasibly see its concepts being applied to marketing analytics and online advertising. While we were not successful at achieving our original goal, this project was overall a useful learning experience.

Code · Dataset

ROS2 Autonomous Maze Solver



TLDR: I spent some time working with a partner on a software package to solve a simple maze. We created a ROS2/Python package to solve the maze by using machine learning and ROS2 NAV2. A simple state machine controlled behaviors and PID controllers enabeled precise movements. Our final submission achieved 90% sign classification accuracy, and was able to complete the maze properly and without error in 5/6 tests.

The goal of our project was to develop a ROS2 software package for a TurtleBot3 that could solve a simple maze. Each wall of the maze had a sign that pointed in the direction the robot should go to solve the maze. Signs were spaced apart such that the maze could be generalized as gird made up of 3'x3' cells. We had control of the robot's initial orientation, but once the robot was placed it had to complete the maze fully autonomously. We discussed several ideas to make this work, including a bug algorithm based design, a simple "always turn right" strategy, or a ROS2 Nav2 based package.

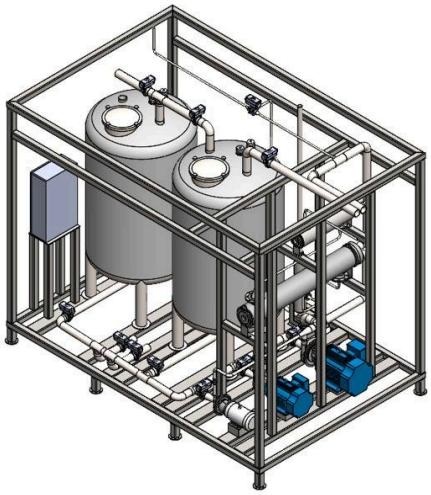
We selected the Nav2 based package because it gave us the most versatility. We created a simple state machine to control the behavior of the robot and trained a KNN classifier for the signs. The robot defaulted to

a forward motion if the path was not obstructed and only tried classification when the forward path was obstructed. If classification was successful, a Nav2 rotational waypoint would be placed in the direction of the next target cell (left, right, or backwards). Once the new heading was reached, forward motion would resume. If classification failed, fallback procedures were created. The robot could adjust its distance to the sign to attempt a better reading. If it couldn't get a better reading within a certain number of attempts, it would default to turning left (the optimal guess given the structure of the maze). A simple PID controller for the wheel output velocity was used as a low level controller.

Our final submission achieved 90% sign classification accuracy, and was able to complete the maze properly and without misclassification in 5/6 tests from multiple starting locations. Given more time, we would try to imrpove the accuracy of our KNN model under different lighting conditions, and would further try to reduce the movement error in the system.

Code

Fluor CIP Skid



TLDR: I worked with a team of seven other Clemson ME students to design a Clean-In-Place Skid system in partnership with Fluor. I focused heavily on the conceptual design side of the project, doing much of my work in the areas of CAD design and thermodynamic calculations. I also acted as the Group Liason, ensuring useful and professional communcation with the Fluor sponsors and and Clemson advisors. Our final design came in second at a multi university showcase.

The goal of this project was to a design a Clean-In-Place Skid system to specifications laid out by the Fluor Corporation. I worked closely with a team of seven other mechanical engineering students to produce both a conceptual design and a prototype design. Most of my work was on the conceptual design, focusing on CAD modeling, thermodynamic calculations, and component research. The design we submitted came in second at a showcase for five different teams from different schools also doing the same project, only barely behind one other team from Clemson.



Conceptual Design

Our conceptual system was required to comply with all FDA and ASME standards while being a CIP system that could be feasibly implemented in a factory. We focused on versatility, with the intent that the system could be used across a wide variety of factories. As such, we utilised a three pump system to maintain efficient performace across a range of flow rates without exceeding the optimal operating ranges for any one pump. Both the rinse tank and recovery tanks were 1000 gallons to allow for the cleaning of large systems. We were given steam as a system inpit so to improve the system's efficiency, we chose to use two heat exchangers. One heat exchanger acted as the primary heat exchanger, utilizing the energy from the first phase change of the steam to heat the working fluid. The then condensed steam then passed through a steam trap to cause a pressure drop, and then through a secondary

heat exchanger to extract the energy from an additional phase change. This allowed us to fully raise the temperature of the working fluid in fewer total cycles, at the price of a higher system cost. Control would be performed by a PLC, as with many other CIP systems.

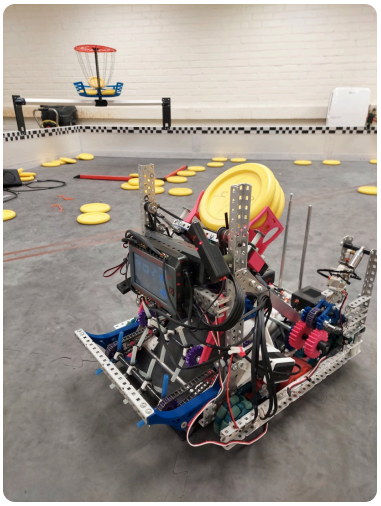
Prototype Design

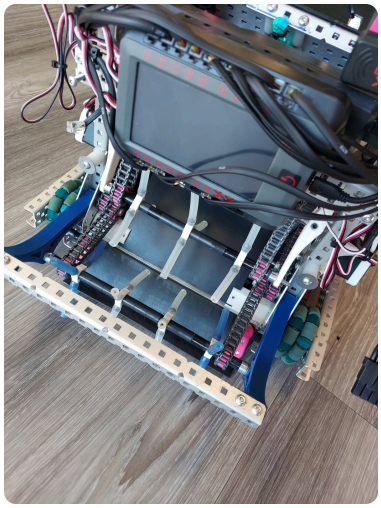


Our prototype design was meant to mimic out conceptual design, without meeting the rigorous requirements for real pharmaceutical CIP systems To replace the conceptual dual pump system, we used a set of flow limiter valves that we could switch between, mimicing the range of flow rates enabled by having multiple pumps. Electric heat exchangers were used rather than steam, since steam was not available for the prototypes. This mimiced the two heat exchanger system used in our conceptual design. A transformer was used to account for the voltage differences between the heat exchangers and the rest of the system. A PLC controlled the entire system, autonomously operating all valves and relays necessary to run a complete cycle. Our design successfully achieved two flow rates and raised the temperature of the input water by about five degrees Farenheit.

VEXU Robot "Meowth"

TLDR: This project is my most mechanically advanced project to date. I worked closely with my team and designed this over the course of an entire school year. It includes numerous subsystems, such as the drivetrain, the intake, the linear-launcher, and the winch release. Perhaps the highlight of the build is the winch release system. Conventional slipgear systems would not work, because multiple full revolutions were required to fully retract the linear launcher, so my team came up with a creative solution: a release arm that physically seperated the gears to avoid motor damage. The linear launcher system that my team and I developed gave the robot a near perfect disk grouping that minimized the number of missed disks, and qualified our team for the World Championships. See the VEX Robotics Spin Up game reveal for more information.



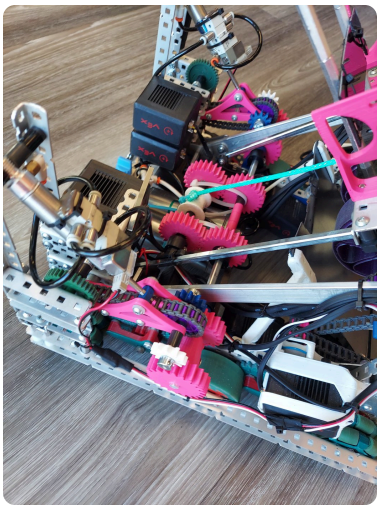


The goal of this project was to create a robot specifically for the Skills (individual robot) challenge of the VEXU SpinUp game, where robots had to launch discs into goals, much like frisbee golf. Robots were to obtain the discs from the ground and were not allowed to hold more than three at any given time. Much of my rolewas to design and test mechanical components and ensure smooth integration with the programming team. My team successfully qualified for the VEX Robotics World Championships this year, along with our head-to-head match sister teams.

Hardware

Most robot designs this year used flywheel launcher systems, because they could be loaded and launched quickly, with less emphasis on accuracy and more emphasis on volume. Since the skills challenge had a lower time limit (1

minute) and did have the concerns about defense from an opposing team, accuracy was paramount. We decided to use a linear slider launcher to maintain high accuracy and keep disc groupings tight.



The primary issue with the linear launcher method was how to avoid motor back spin. A direct linkage would require the launcher elastics to have to spin the motors in reverse to launch, slowing launch velocity and potentially damaging the electronics. Many similar catapult style systems used slip-gears, or gears missing teeth, but this did not give us fine control over when we launched the disks. We instead decided on a pneumatic lever release system, which could physically separate the gears connecting the motor to the winch. Doing so gave us fine control over the launch while avoiding motor back spin. Any issues with re-meshing the gears could be avoided by having gears with large teeth.

The intake system was a linked horizontal intake system. This allowed the robot to easily pick up the flat discs from the floor and deposit them directly into the basket of the linear launcher. Multiple tiers of intake were created, such that the lowest tiers spun the fastest, and the higher tiers spun the slowest, to prevent overshooting the basket while the robot was moving.

Sensing and Programming

All Vex V5 motors come preequipped with encoders, which we supplemented with an IMU. We implemented odometry and PD control to ensure movement accuracy when performing autonomous runs. Since VEX fields are relatively small and VEX's GPS sensors are generally unreliable, autonomous runs relied on dead reckoning to move around. The launcher mechanism was soft-limited to a paricular draw distance in the code to prevent over drawing. Various preset draw distances were allowed so human operators could quickly select how much power they wanted to use.